

C++ para Gameplay Programmer

Teoría esencial — C++ estándar, sin motor

Cómo usarlo: cada tema tiene la idea, la sintaxis y la trampa típica. Está en orden de estudio.

Parte 1: imprescindible en cualquier estudio. **Parte 2:** librería estándar, más importante en estudios sin Unreal.

Las clases de los ejemplos (Fighter, Enemy, SaveData ...) son inventadas.

Parte 1 — Imprescindible en cualquier estudio

1. Stack y heap

Stack: variables locales. Se crean al declararlas y se destruyen solas al llegar al `}` de su scope. Rápido.

Heap: memoria que pides tú con `new`. Vive hasta que alguien hace `delete`.

```
void Foo()
{
    int Health = 100;           // stack: muere al llegar a }
    Enemy* Boss = new Enemy(); // heap: vive hasta el delete
    delete Boss;               // si falta -> memory leak
}
```

Regla: prefiere el stack. Usa el heap solo si el objeto tiene que sobrevivir a su scope, si su tamaño se decide en runtime o si usas polimorfismo.

Trampa: cada `new` sin su `delete` es un leak. En C++ moderno casi nunca escribes `new/delete`: usas smart pointers (temas 9 y 10).

2. Punteros

Un puntero es una variable que guarda una **dirección de memoria**. `nullptr` = no apunta a nada.

```
int Health = 100;
int* Ptr = &Health; // & = "la dirección de"
*Ptr = 50;           // * = "el valor que hay en esa dirección" -> Health = 50

Enemy* E = FindEnemy();
if (E != nullptr) // comprobar siempre antes de usar
{
    E->TakeDamage(10); // -> es lo mismo que (*E).TakeDamage(10)
}
```

Trampa: puntero colgante (dangling): apunta a algo que ya se destruyó. Usarlo es comportamiento indefinido. Ejemplo típico: guardas un puntero a un elemento de un `vector` y luego haces `push_back`; el `vector` puede recolocar sus elementos en otra zona de memoria.

3. Referencias

Una referencia es un **alias**: otro nombre para una variable que ya existe. No puede ser null y no se puede reapuntar.

```
void Heal(int& Health) { Health += 10; } // modifica el original

int PlayerHealth = 90;
Heal(PlayerHealth); // PlayerHealth = 100

int& Alias = PlayerHealth; // obligatorio inicializarla al declararla
```

- **Referencia:** cuando el objeto seguro que existe.
- **Puntero:** cuando puede no existir (`nullptr`) o necesitas cambiar a qué apunta.

Trampa: devolver una referencia a una variable local:

```
int& Bad()
{
    int X = 5;
    return X; // X muere al salir -> la referencia apunta a basura
}
```

4. Paso de parámetros

```
void F(Item I); // copia. Para tipos pequeños: int, float, bool, punteros
void F(Item& I); // el original, puede modificarlo
void F(const Item& I); // el original, solo lectura, sin copia. Default para clases y std::string
void F(Item* I); // puede ser nullptr: parámetro opcional
void F(std::unique_ptr<Item> I); // la función se queda con la propiedad
```

5. const

```
const int MaxHealth = 100; // no puede cambiar

float GetHealth() const; // método que promete no modificar el objeto
void Print(const Item& I); // parámetro de solo lectura

const int* A; // no puedo cambiar el valor a través de A; A sí puede apuntar a otro sitio
int* const B = &X; // B no puede apuntar a otro sitio; el valor sí puede cambiar
```

Truco para leer punteros con const: lee de derecha a izquierda. `const int* A` = "A es un puntero a un int const".

Regla: todo lo que no deba modificar algo, márcalo `const`. El compilador se convierte en tu red de seguridad.

Trampa: sobre un objeto `const` (o recibido por `const&`) solo puedes llamar a métodos `const`. Si olvidas el `const` en un getter, no podrás usarlo.

Trampa: un método `const` puede devolver un puntero no `const`. `const` protege el objeto, no a lo que apuntan sus punteros.

6. Clases

```
class Fighter
{
public:
    Fighter(const std::string& name, float health)    // constructor
        : Name(name)                                // lista de inicialización
        , Health(health)
    {
    }

    ~Fighter() { }                                    // destructor: se ejecuta solo al morir

    float GetHealth() const { return Health; }

private:
    std::string Name;
    float Health = 100.0f;                            // valor por defecto
};

Fighter F("Wolf", 100.0f);    // objeto en el stack
```

- `public`: accesible desde fuera. `private`: solo la propia clase. `protected`: la clase y sus hijas.
- `struct` = `class` pero `public` por defecto. Convención: `struct` para datos simples, `class` para objetos con comportamiento.
- La lista de inicialización construye los miembros directamente. Asignar dentro del cuerpo los construye primero y luego los reasigna. Es obligatoria para miembros `const` y referencias.

Trampa: los miembros se inicializan en el orden en que están **declarados en la clase**, no en el orden de la lista.

Trampa: un `int`, `float`, `bool` o puntero miembro sin inicializar contiene basura. Dale siempre valor por defecto.

Trampa: un constructor de **un solo parámetro** también sirve como **conversión automática**: el compilador lo usa por su cuenta, sin avisar, para convertir ese tipo en tu clase.

```
Weapon(float damage);        // SIN explicit
void Equip(const Weapon& w);

Equip(50.0f);                 // compila: el compilador se inventa un Weapon temporal
```

`explicit` prohíbe esa conversión automática. Si la quieres, la escribes tú a la vista:

```
explicit Weapon(float damage);

Equip(50.0f);                // ERROR
Equip(Weapon(50.0f));        // OK
```

Regla: pon `explicit` en todo constructor de un parámetro, salvo que quieras esa conversión a propósito (como `std::string`, que permite `std::string S = "hola";`).

7. Sobrecarga de operadores

Puedes definir qué hacen `+`, `==`, `<<` ... con tus clases. Muy usado en matemática de juegos (vectores).

```
struct Vec2
{
    float X = 0.0f;
    float Y = 0.0f;

    Vec2 operator+(const Vec2& other) const { return { X + other.X, Y + other.Y }; }
    Vec2 operator*(float s) const           { return { X * s, Y * s }; }
    bool operator==(const Vec2& other) const { return X == other.X && Y == other.Y; }
};

Vec2 Pos = Vec2{ 1.0f, 2.0f } + Vec2{ 3.0f, 4.0f }; // { 4, 6 }
```

Trampa: comparar floats con `==` es mala idea por errores de redondeo. En juegos se compara con un margen: `std::abs(A - B) < 0.0001f`.

8. RAI

El recurso se pide en el **constructor** y se libera en el **destructor**. Como el destructor se ejecuta solo al morir el objeto, el recurso se libera solo, aunque haya `return` tempranos.

Idea clave: **el recurso vive lo mismo que el objeto que lo guarda.**

```
class ScopedSave
{
public:
    explicit ScopedSave(const char* path) : Data(new SaveData(path)) {} // adquiere
    ~ScopedSave() { delete Data; } // libera
    SaveData* Get() const { return Data; }

private:
    SaveData* Data;
};

void LoadGame()
{
    ScopedSave Save("slot1.sav");
    if (Save.Get()->IsCorrupted())
    {
        return; // ~ScopedSave() se ejecuta solo -> sin leak
    }
    // y aquí también
}
```

La librería estándar está hecha así: `std::vector`, `std::string`, `std::unique_ptr`, `std::lock_guard` ...

Trampa: si copias un `ScopedSave`, las dos copias apuntan al mismo `SaveData` y las dos harán `delete` (double delete). Solución: prohibir la copia (tema 12) o usar `unique_ptr`.

9. unique_ptr

Puntero con **un único dueño**. El dueño es el responsable de borrar. Cuando el `unique_ptr` muere, borra el objeto.

```
#include <memory>

std::unique_ptr<Fighter> A = std::make_unique<Fighter>("Wolf", 100.0f);
A->TakeDamage(10);

std::unique_ptr<Fighter> B = A;           // ERROR: no se puede copiar
std::unique_ptr<Fighter> C = std::move(A); // OK: C es el nuevo dueño, A queda vacío

Fighter* Raw = C.get(); // puntero normal para usarlo, sin soltar la propiedad
if (C) { }              // ¿tiene algo dentro?
C.reset();              // borra el objeto ya
```

- El dueño guarda el `unique_ptr`. Los demás usan `T*` o `T&` ("solo lo uso, no lo borro").
- Si una función se va a **quedar** con él, recíbelo **por valor**: `void Add(std::unique_ptr<T> item)`.
- `std::vector<std::unique_ptr<T>>`: el vector es el dueño. Si el vector crece, se mueven los `unique_ptr`, no los objetos, así que los `T*` que repartiste siguen siendo válidos.

Trampa: usar un `unique_ptr` después de moverlo: está vacío (`nullptr`).

10. shared_ptr y weak_ptr

shared_ptr: varios dueños. Lleva un contador; el objeto se borra cuando muere el último `shared_ptr`.

weak_ptr: observa sin ser dueño. No mantiene vivo el objeto. Para usarlo, `lock()`.

```
std::shared_ptr<Texture> T1 = std::make_shared<Texture>();
std::shared_ptr<Texture> T2 = T1; // contador = 2
T1.use_count();                  // 2

std::weak_ptr<Texture> W = T1;    // no suma al contador

if (std::shared_ptr<Texture> Locked = W.lock()) // vacío si el objeto ya murió
{
    Locked->Bind();
}
```

Regla: por defecto `unique_ptr`. `shared_ptr` solo si la propiedad es realmente compartida. `weak_ptr` para "lo miro pero no lo mantengo vivo" (ejemplo: el objetivo de un enemigo).

Trampa: ciclo: A tiene un `shared_ptr` a B y B a A. El contador nunca llega a 0 y ninguno se borra (leak). Rompe el ciclo con `weak_ptr`.

Trampa: `shared_ptr` tiene coste (contador compartido entre hilos). No lo uses "por si acaso".

11. Herencia y virtual

```
class Fighter
{
public:
    virtual ~Fighter() = default;           // destructor virtual: obligatorio si hay herencia
    virtual void Attack(Fighter& target);    // los hijos pueden reemplazarla
    virtual void SpecialMove() = 0;         // virtual pura: los hijos DEBEN implementarla
};                                           // -> Fighter es abstracta: no se puede instanciar

class Samurai : public Fighter
{
public:
    void Attack(Fighter& target) override;  // override: el compilador comprueba que existe en
    void SpecialMove() override;           // el padre
};

std::unique_ptr<Fighter> F = std::make_unique<Samurai>();
F->Attack(Other);                          // llama a Samurai::Attack (polimorfismo)
```

- **Con `virtual`:** se llama a la versión del objeto real, aunque el puntero sea del padre. Se decide en runtime.
- **Sin `virtual`:** se llama a la versión del tipo del puntero.
- Llamar a la versión del padre desde el hijo: `Fighter::Attack(target);`
- Si no hay relación "es un", mejor composición (tener un miembro) que herencia.

Trampa: destructor sin `virtual` y borrar un hijo a través de un puntero al padre: solo se ejecuta el destructor del padre (comportamiento indefinido).

Trampa: sin `override`, un error en la firma crea una función nueva en vez de reemplazar la del padre, sin avisar. Pon siempre `override`.

Trampa: slicing: copiar un hijo en una variable del padre **por valor** corta la parte del hijo. Con clases polimórficas, trabaja con punteros o referencias.

```
Samurai S;
Fighter F = S;    // (si Fighter no fuera abstracta) F es solo un Fighter: lo de Samurai se pierde
```

12. Copia y move

Toda clase tiene 5 funciones especiales. Si no las escribes, el compilador las genera solo:

```
class Buffer
{
public:
    ~Buffer();           // destructor
    Buffer(const Buffer& other); // constructor de copia
    Buffer& operator=(const Buffer& other); // asignación por copia
    Buffer(Buffer&& other) noexcept; // constructor de move
    Buffer& operator=(Buffer&& other) noexcept; // asignación por move
};
```

- **Copia:** duplica el contenido. **Move:** se lleva los recursos del otro y lo deja vacío. Mucho más barato.
- `std::move` no mueve nada por sí solo: marca el objeto como "se puede mover" (`&&`). El move lo hacen el constructor o la asignación de move.
- **Regla de 0 (lo ideal):** si tu clase solo tiene miembros que ya se gestionan solos (`vector`, `string`, `unique_ptr`), no escribas ninguna de las 5.
- **Regla de 3/5:** si tienes que escribir una (porque gestionas un recurso a mano), casi seguro necesitas todas.

Prohibir la copia:

```
Buffer(const Buffer&) = delete;
Buffer& operator=(const Buffer&) = delete;
```

Trampa: una clase con un puntero crudo dueño y sin regla de 3: la copia por defecto copia el puntero, no el objeto. Resultado: double delete.

13. Lambdas

Función sin nombre que escribes justo donde la usas. Puede **capturar** variables de alrededor.

```
auto IsDead = [](const Fighter& f) { return !f.IsAlive(); };
IsDead(Wolf);                                     // se llama como una función

float MinHealth = 20.0f;
auto IsLow = [MinHealth](const Fighter& f)          // captura por copia
{
    return f.GetHealth() < MinHealth;
};

int Count = 0;
auto Increment = [&Count]() { ++Count; };          // captura por referencia
```

- `[]` nada · `[x]` x por copia · `[&x]` x por referencia · `[=]` todo por copia · `[&]` todo por referencia · `[this]` el objeto actual.
- Para guardarla en un miembro o pasarla a otra función: `std::function<bool(const Fighter&)>` (`#include <functional>`).

```
std::function<void(int)> OnDamaged; // un "callback"
OnDamaged = [](int amount) { std::cout << amount << "\n"; };
OnDamaged(25);
```

Trampa: capturar por referencia (o `this`) y ejecutar la lambda más tarde, cuando esa variable u objeto ya ha muerto: dangling. Típico en callbacks y timers.

14. Templates

Código que funciona con cualquier tipo. El compilador genera una versión por cada tipo que uses.

```

template <typename T>
T Clamp(T value, T min, T max)
{
    if (value < min) return min;
    if (value > max) return max;
    return value;
}

Clamp(150, 0, 100);           // T = int    -> 100
Clamp(1.5f, 0.0f, 1.0f);     // T = float  -> 1.0f
Clamp(1.5f, 0, 1);           // ERROR: T no puede ser float e int a la vez

template <typename T>
class Pool
{
public:
    void Add(const T& item) { Items.push_back(item); }
private:
    std::vector<T> Items;
};

Pool<int> Numbers;

```

Trampa: los templates se escriben enteros en el `.h`. El compilador necesita ver el código al usarlos. Si los defines en el `.cpp`: error de linker.

Trampa: los errores de templates son larguísimos. Lee la primera línea del error: ahí suele estar la causa.

15. enum class y struct

```

enum class EState { Idle, Attacking, Blocking, Dead };

EState State = EState::Idle;

switch (State)
{
    case EState::Idle:      /* ... */ break;
    case EState::Attacking: /* ... */ break;
    default:                break;
}

int AsInt = static_cast<int>(State); // no se convierte a int sola

struct HitInfo
{
    float Damage = 0.0f;
    bool bIsCritical = false;
};

HitInfo Hit{ 25.0f, true };

```

Usa siempre `enum class` en vez de `enum`: no se convierte solo a `int` y sus nombres no se mezclan con el resto del código.

16. Casts

```
float Speed = 3.7f;
int Rounded = static_cast<int>(Speed);           // 3. Conversiones normales

Fighter* F = GetFighter();
Samurai* S = dynamic_cast<Samurai*>(F);           // nullptr si no es un Samurai
if (S != nullptr) { }                           // (requiere que la clase tenga algo virtual)
```

- Evita el cast estilo C `(int)Speed`: hace lo que sea sin avisar.
- `const_cast` y `reinterpret_cast`: casi nunca en gameplay.

17. static

```
class Fighter
{
public:
    static int Count;           // uno solo, compartido por todos los Fighter
    static void PrintCount();   // se llama sin objeto: Fighter::PrintCount();
};

int Fighter::Count = 0;        // se define en el .cpp

int NextId()
{
    static int Id = 0;         // se inicializa una sola vez y conserva su valor entre
    llamadas                   llamadas
    return ++Id;
}
```

Trampa: `static` es estado global. Úsalo con cuidado: es difícil de testear y de depurar.

18. .h, .cpp y compilación

- `.h`: declaraciones (qué existe). `.cpp`: definiciones (cómo funciona).
- `#pragma once` al principio del `.h`: evita que se incluya dos veces.
- **Forward declaration:** `class Fighter;` en el `.h` basta si solo usas `Fighter*` o `Fighter&`. El `#include` completo va en el `.cpp`. Compila más rápido y evita includes circulares.
- Necesitas el `#include` completo si heredas de la clase, la tienes como miembro por valor o llamas a sus métodos.
- **Compilador:** compila cada `.cpp` por separado. **Linker:** une los resultados.
- **Error de compilador:** sintaxis o tipos dentro de un archivo.
- **Error de linker** (`unresolved external symbol`): algo está declarado pero no encuentra su definición. Causas: falta el cuerpo de la función, el `.cpp` no está en el proyecto o la firma no coincide entre `.h` y `.cpp`.

Trampa: una clase con un miembro `std::unique_ptr<T>` y `T` solo forward-declarado: el destructor de la clase necesita ver `T` completo. Si da error, declara el destructor en el `.h` y defínelo en el `.cpp` con `= default`.

19. Comportamiento indefinido (UB)

El estándar no dice qué pasa. Puede funcionar, crashear o corromper memoria en silencio y fallar mucho después. Los típicos:

- Usar un puntero colgante o un `nullptr`.
- Leer fuera de un array. `vector[i]` no comprueba nada; `vector.at(i)` sí.
- Leer una variable sin inicializar.
- Hacer `delete` dos veces, o de algo que no vino de `new`.
- Borrar un hijo a través de un puntero al padre sin destructor `virtual`.
- Desbordar un `int` con signo.
- Añadir o borrar elementos de un `vector` mientras lo recorres con un range-for.

Parte 2 — Librería estándar (sobre todo fuera de Unreal)

20. Contenedores

- `std::vector<T>`: array dinámico, memoria contigua. **El que usas por defecto.** Acceso $O(1)$, `push_back` $O(1)$, insertar o borrar en medio $O(n)$.
- `std::array<T, N>`: tamaño fijo, conocido al compilar.
- `std::unordered_map<K, V>`: tabla hash. Buscar $O(1)$ de media. Sin orden.
- `std::map<K, V>`: árbol ordenado por clave. Buscar $O(\log n)$.
- `std::unordered_set<T>` / `std::set<T>`: como los map, pero solo claves (sin repetidos).
- `std::string`: texto.

```
std::vector<int> V = { 1, 2, 3 };
V.push_back(4);
V.size();           // 4
V[0];               // 1
V.reserve(100);     // reserva memoria de golpe: evita recolocaciones

std::unordered_map<std::string, int> Ammo;
Ammo["Arrows"] = 20; // inserta o sobrescribe

auto It = Ammo.find("Bombs");
if (It != Ammo.end())
{
    It->first;           // clave
    It->second;          // valor
}
```

Trampa: `map["clave"]` **inserta** la clave si no existe. Para solo consultar, usa `find`.

Trampa: `push_back` puede recolocar el vector e invalidar punteros, referencias e iteradores a sus elementos.

¿Por qué `vector` casi siempre? Memoria contigua = la CPU la lee muy rápido (caché). Una lista enlazada suele ser más lenta en la práctica aunque en teoría inserte en $O(1)$.

21. Range-for e iteradores

```
for (const Fighter& f : Fighters) { } // leer sin copiar
for (Fighter& f : Fighters) { }       // modificar
for (auto& f : Fighters) { }          // igual, con auto

for (auto It = V.begin(); It != V.end(); ++It)
{
    *It;                               // el elemento
}
```

`begin()` = primer elemento. `end()` = una posición **después** del último. No se desreferencia nunca.

Trampa: `for (Fighter f : Fighters)` sin `&` copia cada elemento.

22. Algoritmos

```
#include <algorithm>

auto It = std::find_if(V.begin(), V.end(),
    [](const Fighter& f) { return f.IsAlive(); });
if (It != V.end()) { } // encontrado

std::sort(V.begin(), V.end(),
    [](const Fighter& a, const Fighter& b) { return a.GetHealth() < b.GetHealth(); });

auto Alive = std::count_if(V.begin(), V.end(),
    [](const Fighter& f) { return f.IsAlive(); });

std::erase_if(V, [](const Fighter& f) { return !f.IsAlive(); }); // C++20: borra los que
cumplan
```

Trampa: `std::remove_if` **no borra**: mueve los elementos al final. Hay que llamar a `erase` después ("erase-remove"). En C++20, `std::erase_if` lo hace todo de una vez.

23. auto

```
auto X = 5; // int
auto It = Ammo.find(Key); // tipos largos: auto ayuda a leer
const auto& First = V[0]; // referencia const, sin copia
```

Trampa: `auto` quita el `const` y el `&`. `auto X = V[0];` hace una **copia**. Si no quieres copia, escribe `const auto&` o `auto&`.

Checklist de entrevista

Si sabes explicar esto con tus palabras y un ejemplo, tienes la base:

- ¿Diferencia entre stack y heap? ¿Qué es un memory leak?
- ¿Puntero o referencia? ¿Cuándo cada uno?
- ¿Qué es un puntero colgante? Pon un ejemplo.
- ¿Para qué sirve `const` en un método? ¿Y en un parámetro `const&`?
- ¿Qué es RAI?
- ¿`unique_ptr`, `shared_ptr` o `weak_ptr`? ¿Cuándo cada uno?
- ¿Qué hace `virtual`? ¿Por qué el destructor de una clase base debe ser `virtual`?
- ¿Qué hace `override`?
- ¿Diferencia entre copiar y mover? ¿Qué hace `std::move`?
- ¿Qué es la regla de 0 / 3 / 5?
- ¿Qué es una lambda y qué significa capturar por referencia?
- ¿Por qué `vector` suele ser más rápido que una lista enlazada?
- ¿Qué es un error de linker y por qué ocurre?